

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

Sensor Systems

Laboratory 03
GPIOs and Interrupts

25.09.2026 / 28.09.2026 I Federica Villa, Arianna De Vecchi

Contents

1. GPIOs

2. EXTI

3. Functions and exercises

1. GPIOs

01

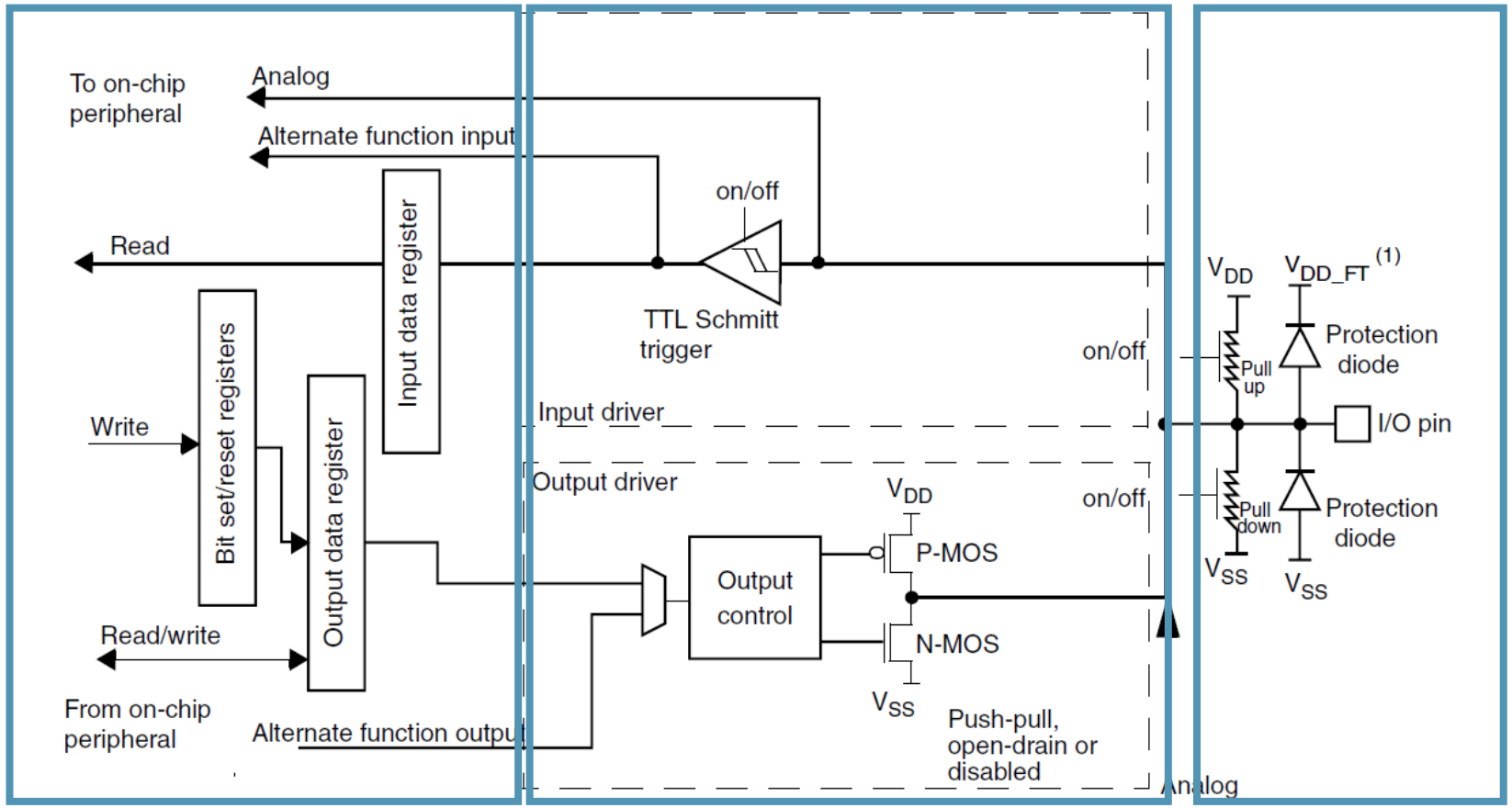
General Purpose Input/Outputs

A GPIO is a **configurable pin** used to read or write digital signals.

Modes:

- **Input** → read external analog or digital signals (e.g., buttons, analog voltages…)
- **Output** → drive analog or digital signals (e.g., LEDs, analog reference voltages…)
- **Alternate Function** → used by peripherals (e.g., UART, SPI, Timers, etc.)

General Purpose Input/Outputs

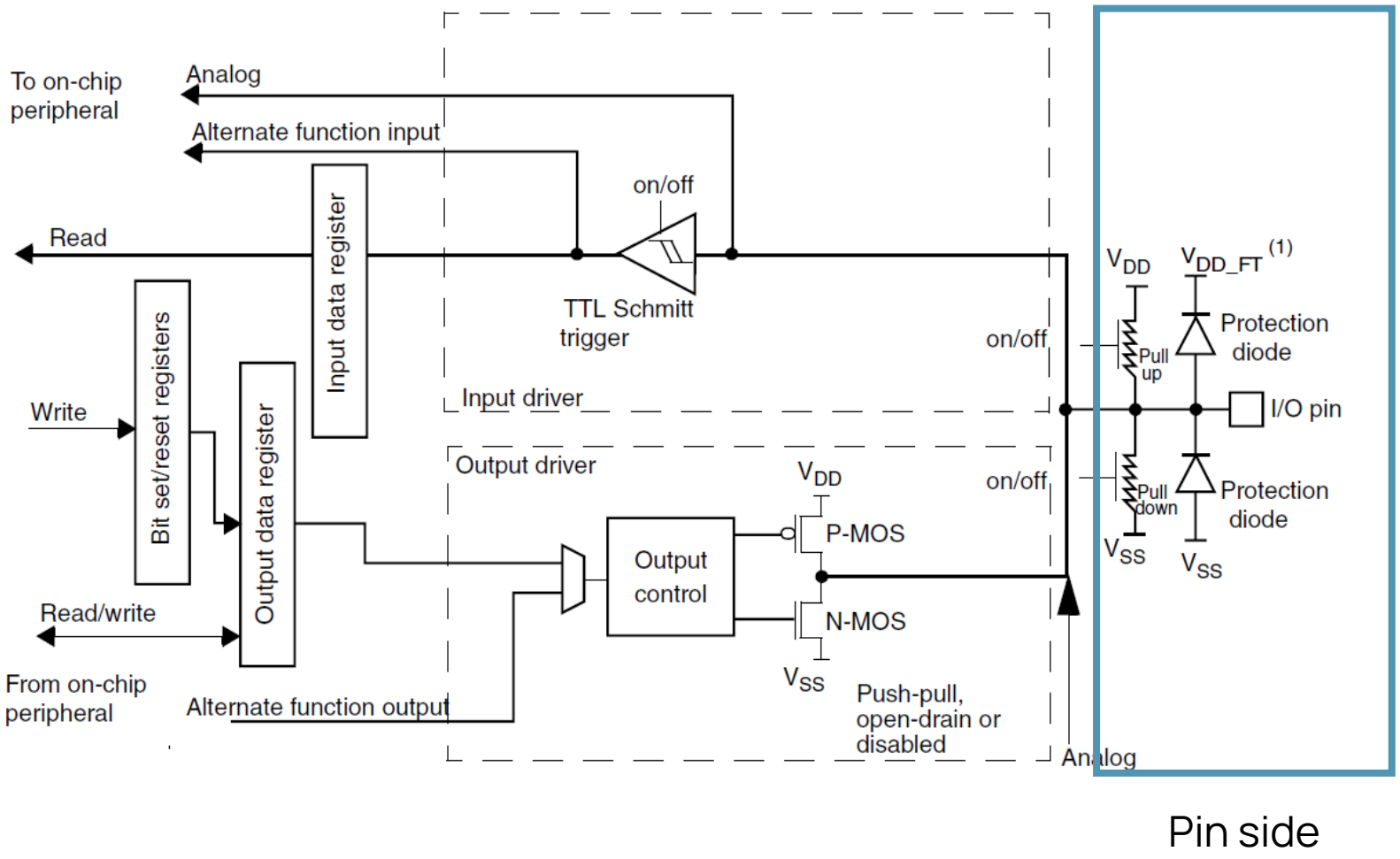


Register/logic side | Input + output driver | Pin side

GPIOs:

- 6 ports (A, …, E, H)
- Each port 16 PINs

General Purpose Input/Outputs



Pin side

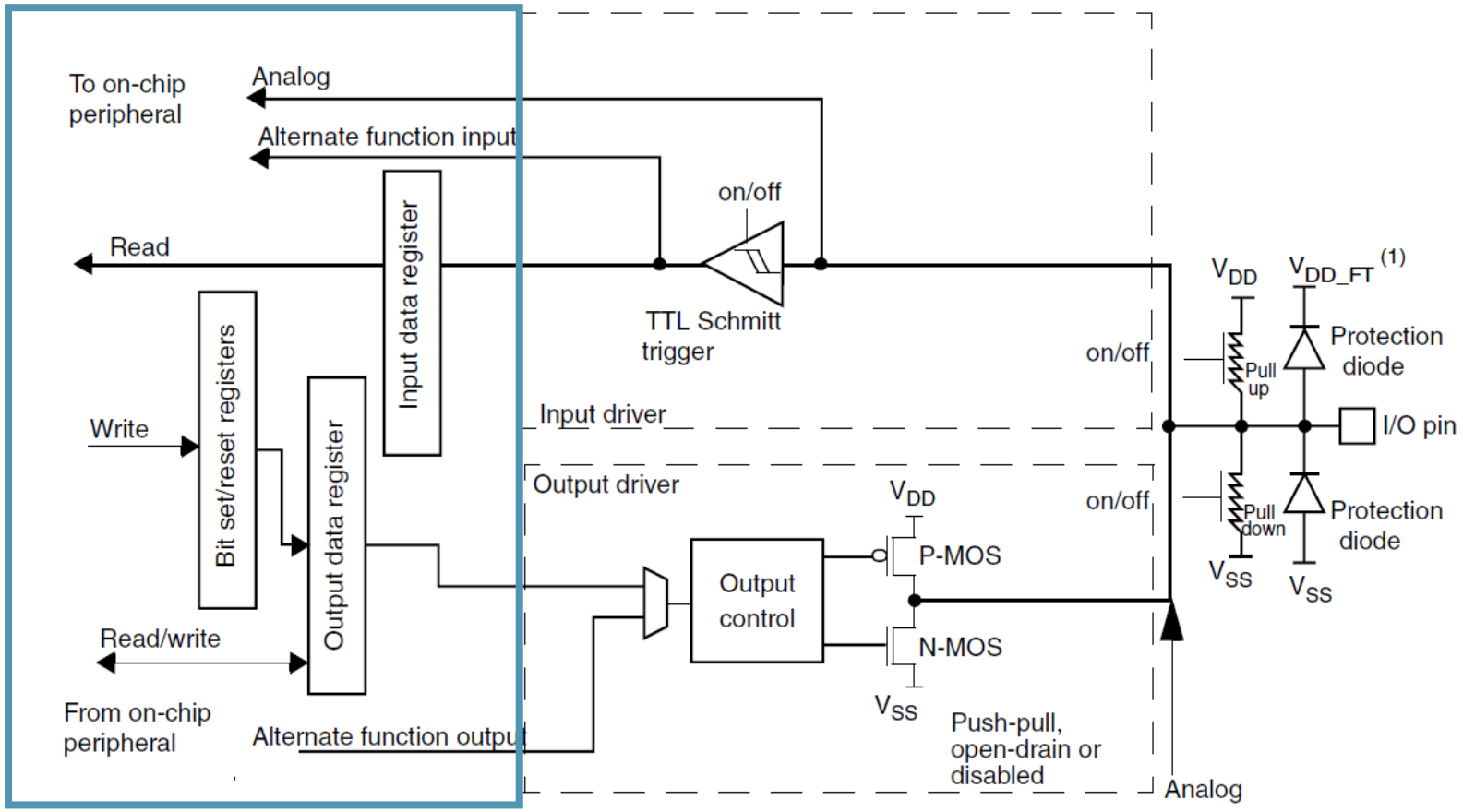
Pin protection and 5V tolerance

- Two clamp diodes to V_DD and V_SS: ESD and overvoltage protection: They bound the voltage that can safely be applied to the pin.

Internal pull-up/pull-down

- Weak resistors (~30–50 kΩ), each with an independent on/off switch.
- Options: floating, pull-up, pull-down (never both at once).
- Use them for: floating lines, push-buttons, open-drain nets.
- Prefer an external resistor when you need a precise value or more current.

General Purpose Input/Outputs

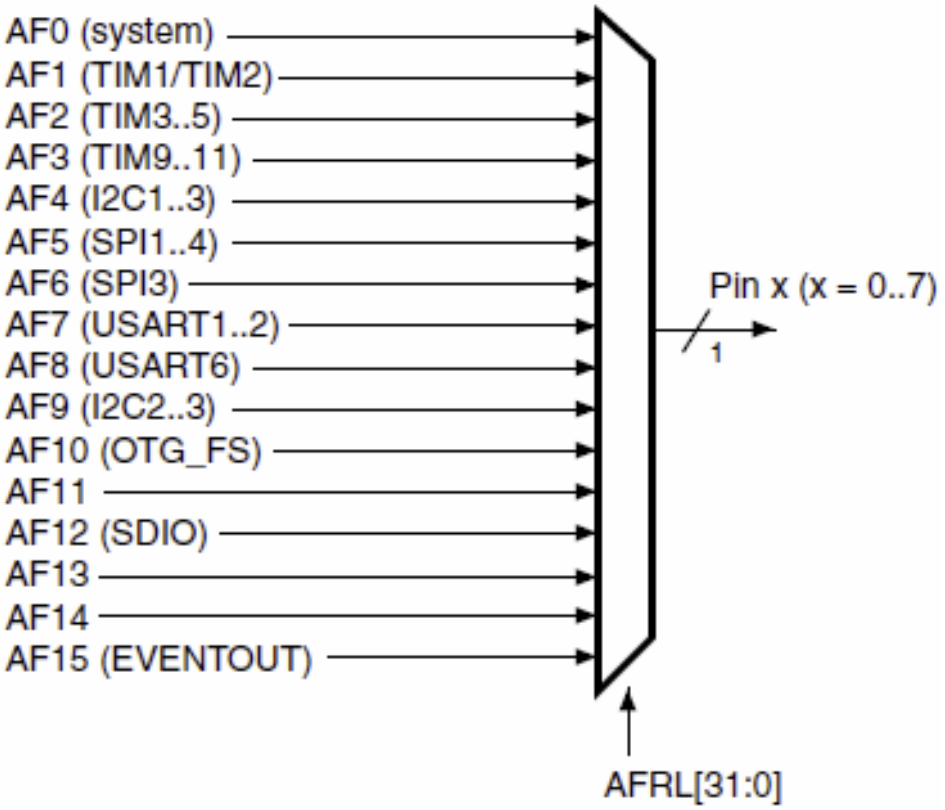


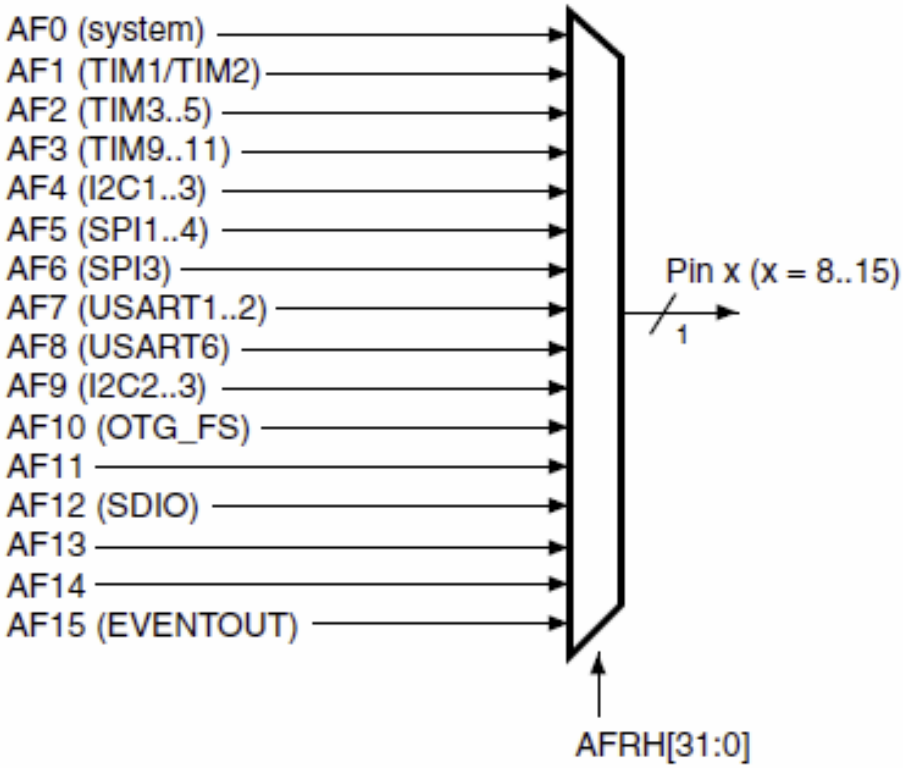
Register/logic side

Register/logic side

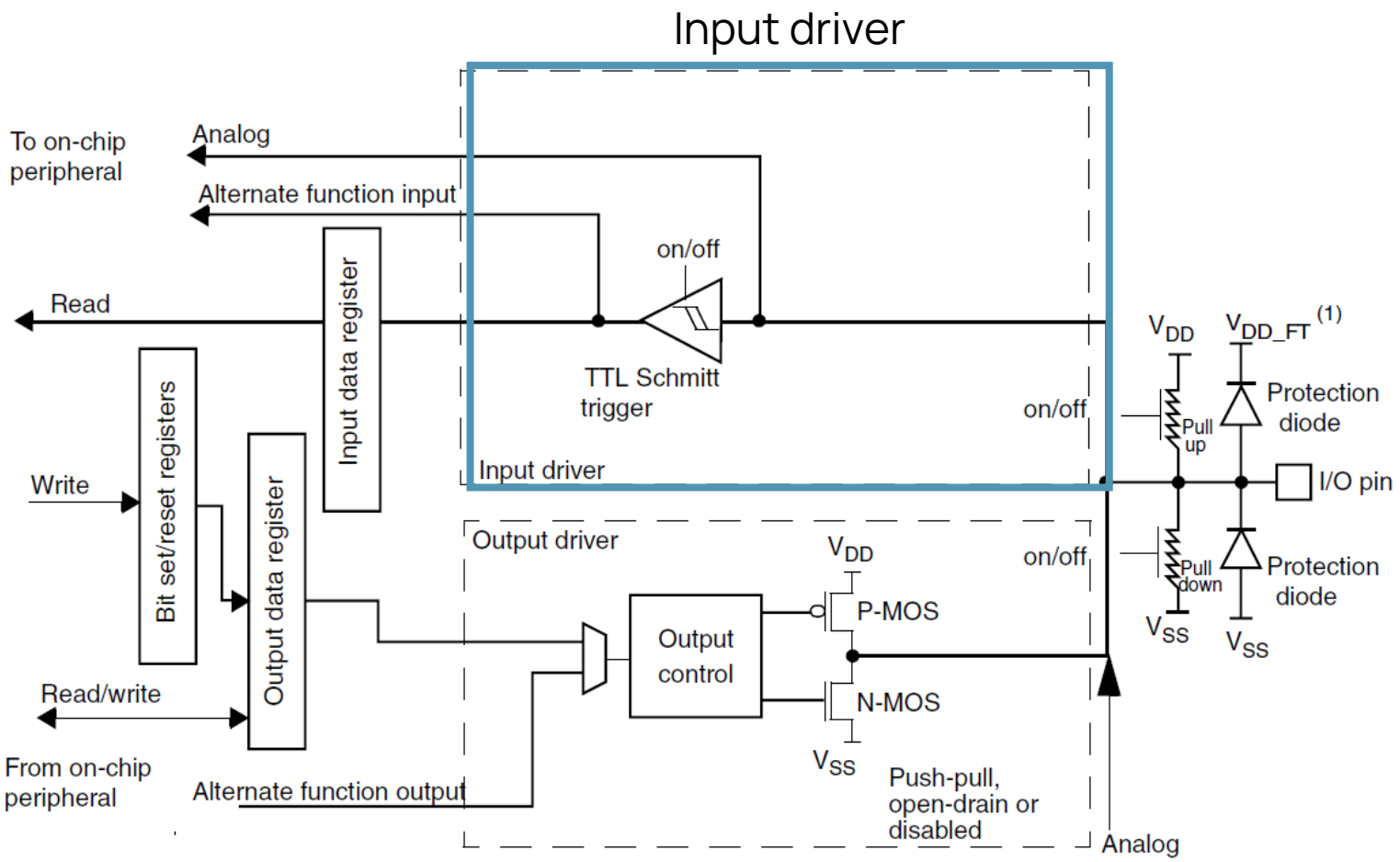
- **ODR**: output data register (what the CPU writes)
- **IDR**: input data register (what the CPU reads)
- **Read/Write/Read-Write buses** link the cell to the core
- **Alternate-function** path connects the pin to an on-chip peripheral
- **BSRR**: atomic single-bit set/reset

Alternate Functions





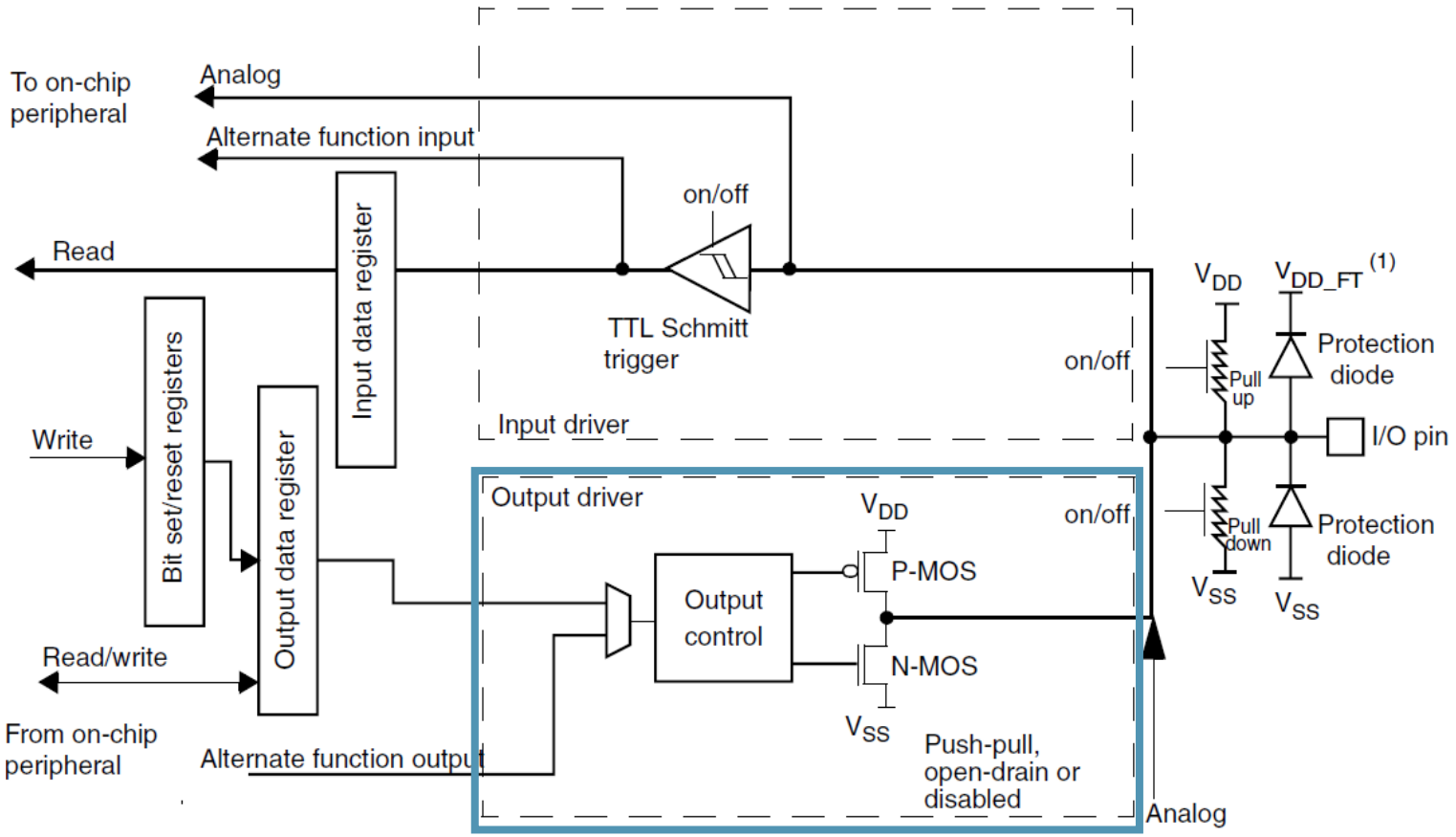
General Purpose Input/Outputs



Input path and Schmitt trigger

- **Schmitt trigger** with hysteresis: two thresholds, V_IL and V_IH
- Purpose: noise immunity and clean edges from slow-rising inputs
- **IDR** captures the **digital** level → Read bus
- **Alternate-function input** feeds a peripheral (e.g. UART RX, timer capture)
- **Analog input** bypasses the digital front-end and goes straight to the analog peripheral (ADC / comparator / DAC)

General Purpose Input/Outputs



Output driver

Output stage

- **MUX** selects the source: ODR (general-purpose) or alternate-function output
- **Output control** drives the two transistors
- **Push-pull**: P-MOS + N-MOS actively drive both 0 and 1
- **Open-drain**: P-MOS off, only N-MOS → needs a pull-up (EX: I^2C)
- **Disabled**: both transistors off

General Purpose Input/Outputs

Open-drain + pull-up

- An open-drain stage can only pull the line low or release it.
- It never drives high, so the pull-up is what creates the logic 1.
- Example: I2C communication

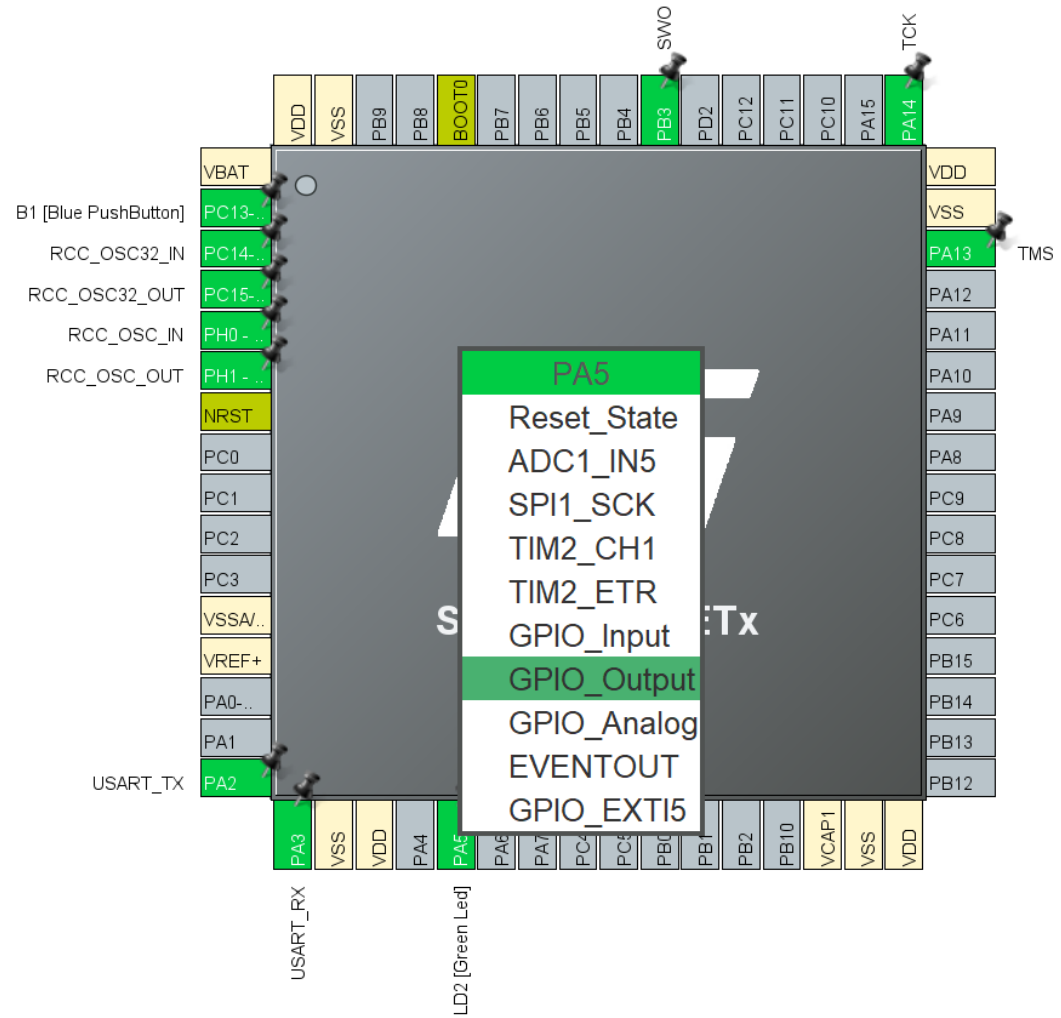
Input + pull

- In input mode there is no output stage, so nothing fixes the pin level.
- An unconnected input floats: it picks up noise, reads random 0/1, and can even draw extra current as the input buffer hovers around its threshold. The pull sets the resting level.
- Example: Push-button to GND + internal pull-up

2. EXTI

02

General Purpose Input/Outputs

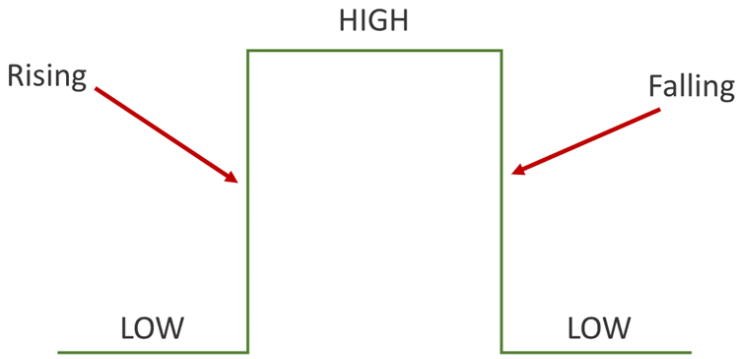


As we discussed last time, each pin can be configured in the GUI.

On some pins, there is an option named **EXTI**: what is it?

External interrupt (EXTI) peripheral

- The **EXTI (External Interrupt)** on an STM32 microcontroller is a peripheral used to handle **interrupts triggered by external events**, such as changes in GPIO pins.
- It allows specific pins to generate interrupts when they detect a **rising edge**, **falling edge**, or **both**.
- Each EXTI line can be connected to a specific GPIO pin, and there are multiple EXTI lines available.
- EXTI lines are connected to the NVIC, which manages the interrupt priorities and enables or disables them globally.



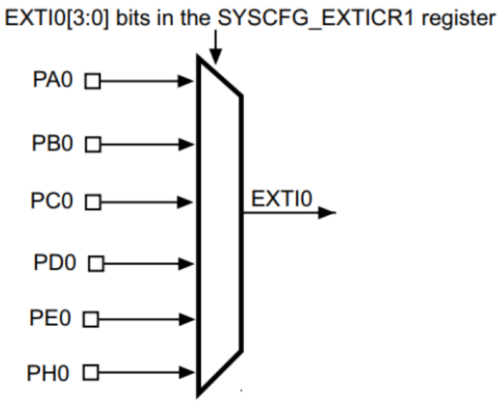
External interrupt (EXTI) peripheral

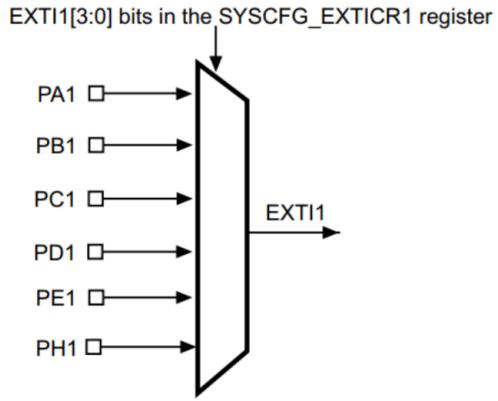
EXTI can manage both **interrupts** and **events**:

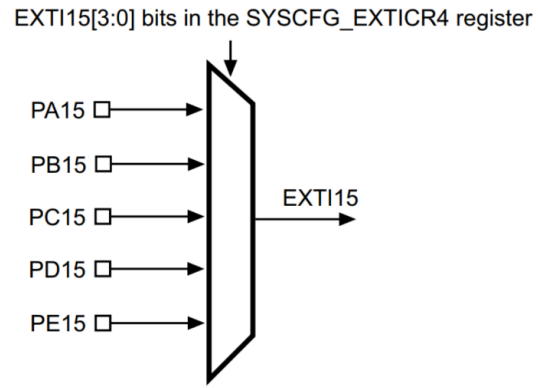
- When an **interrupt** occurs, the CPU is notified, and it immediately pauses its current execution to handle the interrupt by running an Interrupt Service Routine (ISR). After processing, it resumes its previous task.
- An **event** can trigger specific actions in the system (e.g., starting a peripheral or changing internal states) without interrupting the CPU. Events are processed silently, allowing the CPU to continue its current execution unless explicitly handled by software.

External interrupt/event line mapping

Up to **81 GPIOs** (STM32F401xB/C and STM32F401xD/E) are connected to the **16 external interrupt/ event lines** in the following manner. There are only **16 EXTI lines for GPIOs**: line x connects to pin x of one selected port. The SYSCFG_EXTICR register picks which port.







The five other EXTI lines are connected as follows:

- EXTI line 16 is connected to the Power Voltage Detector (PVD) output
- EXTI line 17 is connected to the Real Time Clock (RTC) Alarm event
- EXTI line 18 is connected to the USB On-The-Go Full Speed (OTG FS) Wakeup event
- EXTI line 21 is connected to the RTC Tamper and TimeStamp events
- EXTI line 22 is connected to the RTC Wakeup event

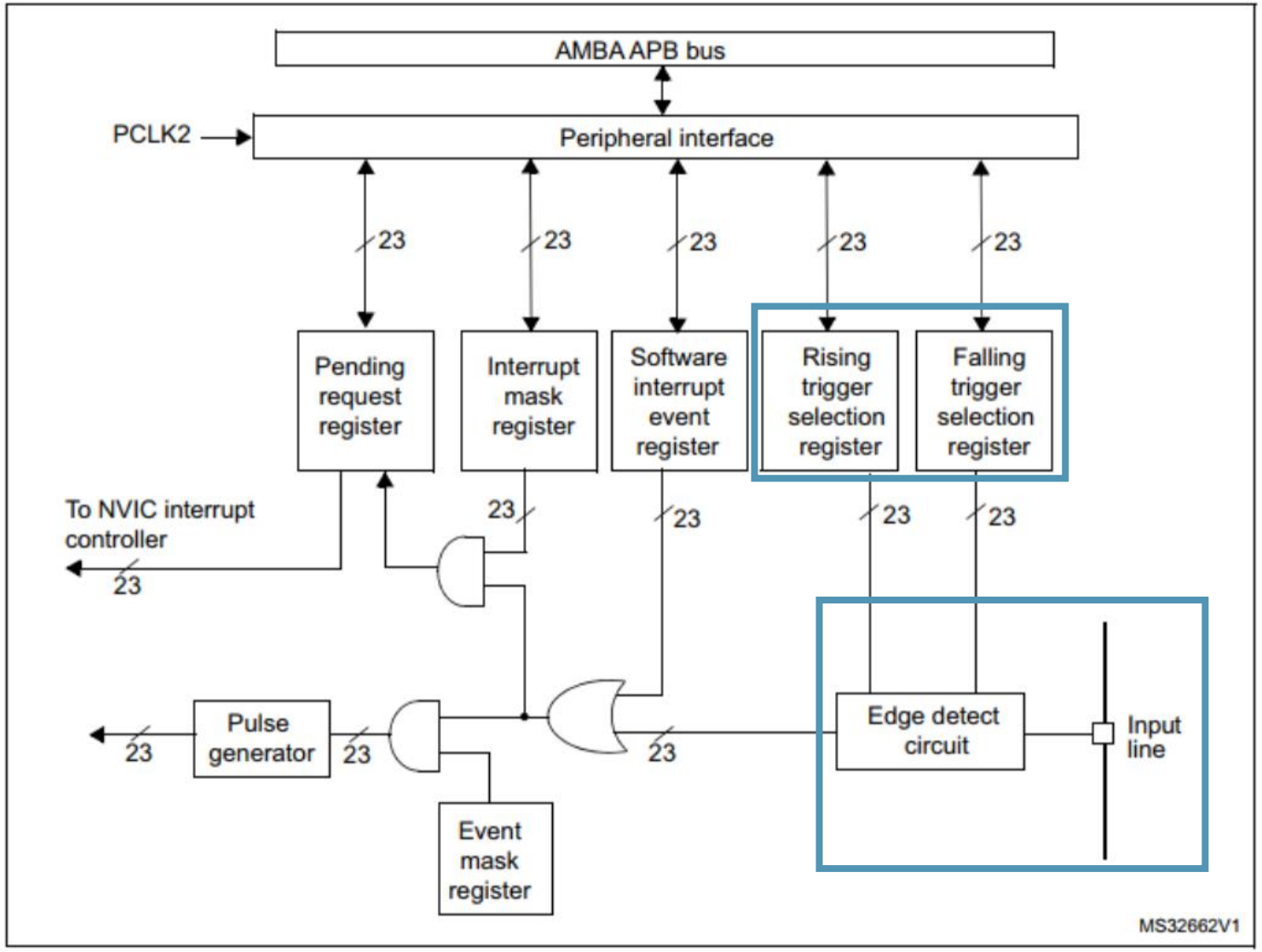
POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

SYSCFG Register map

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	SYSCFG_MEMRMP	Reserved																														MEM_MODE	
	Reset value																															x	x
0x04	SYSCFG_PMC	Reserved															ADC1DC2	Reserved															
	Reset value																0																
0x08	SYSCFG_EXTICR1	Reserved																EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	SYSCFG_EXTICR2	Reserved																EXTI7[3:0]				EXTI6[3:0]				EXTI5[3:0]				EXTI4[3:0]			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	SYSCFG_EXTICR3	Reserved																EXTI11[3:0]				EXTI10[3:0]				EXTI9[3:0]				EXTI8[3:0]			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x14	SYSCFG_EXTICR4	Reserved																EXTI15[3:0]				EXTI14[3:0]				EXTI13[3:0]				EXTI12[3:0]			
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x20	SYSCFG_CMPCR	Reserved																								READY	Reserved						CMP_PD
	Reset value																									0							0

- The **register map** is the table listing, for each register, its address (offset) and what each bit does
- Example: SYSCFG_EXTICR1 (offset 0x08) → bits [3:0] pick the port for EXTI0, [7:4] for EXTI1, …
- Each EXTI line already knows its pin number, only the **port** must be chosen (PA0? PB0? PC0?)
- Choosing 1 of up to 16 ports needs **4 bits** ($2^4 = 16$), even if specific MCUs have less (we have 6)
- These 4 bits live in **SYSCFG_EXTICRx**: 0000 = port A, 0001 = B, …

EXTI block diagram

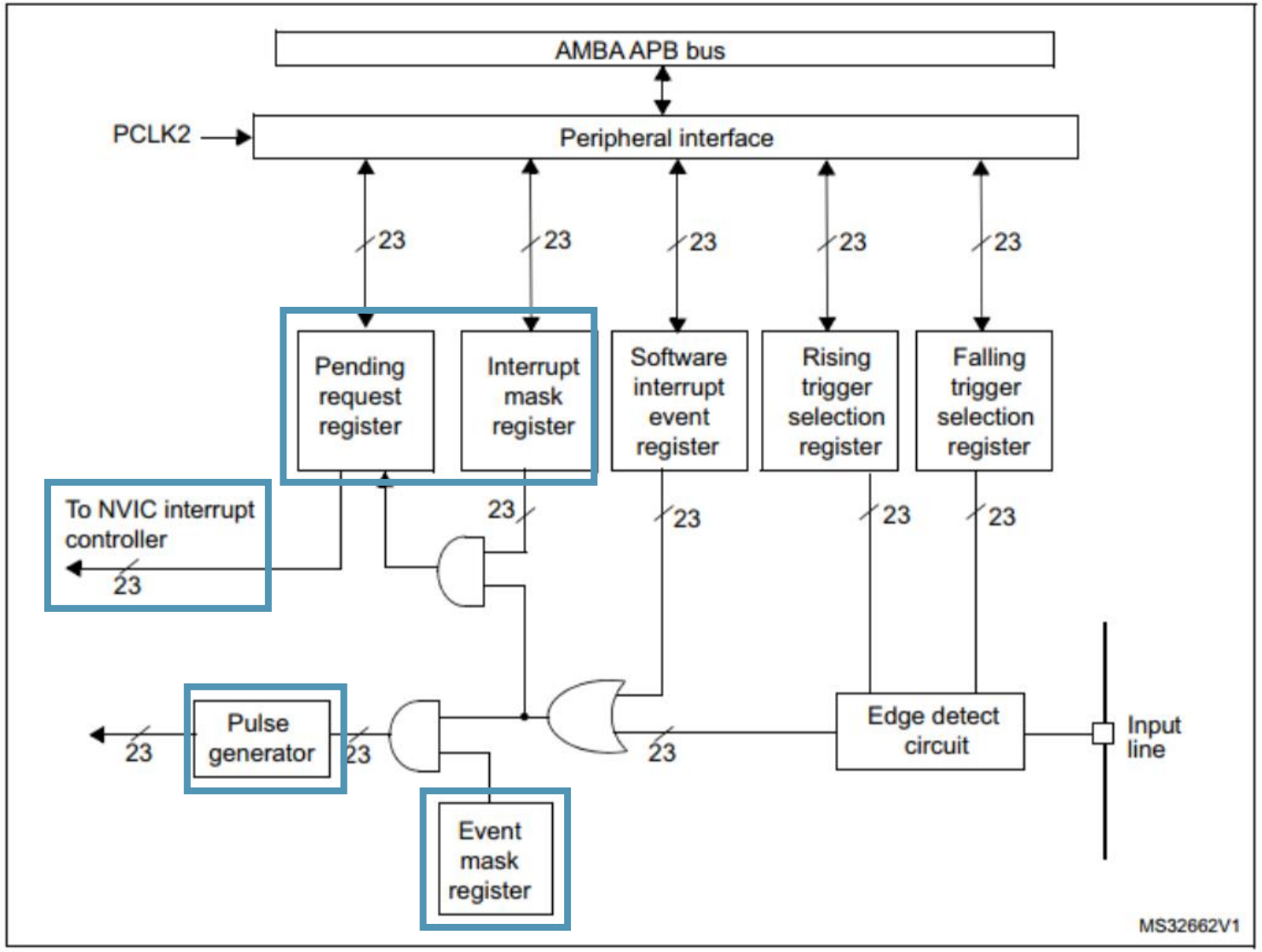


Read the signal right → left: from Input line to CPU/hardware.

Every wire is one structure, replicated for all 23 lines.

- **Edge detect circuit** reacts to a change, not to a level
- Rising edge: 0 → 1
- Falling edge: 1 → 0
- Set by the Rising / Falling trigger selection registers

EXTI block diagram

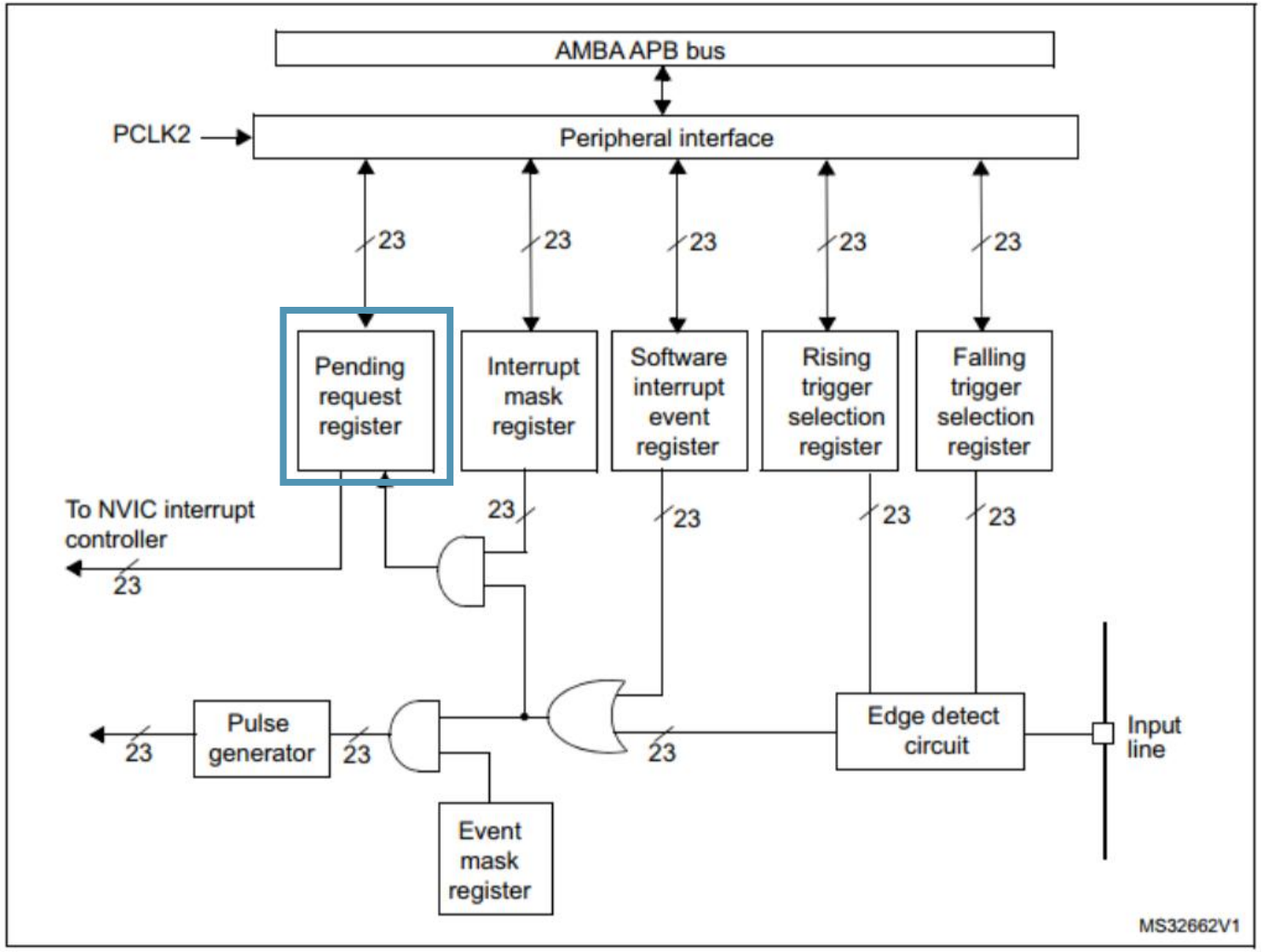


After triggering, the signal splits into two paths:

- **Interrupt path** → through the Interrupt mask register → Pending request register → out to the NVIC (runs the ISR)
- **Event path** → through the Event mask register → Pulse generator (wakes the chip or starts another peripheral, no code)

Each path has its own **mask register** → enable interrupt, event, both, or neither

EXTI block diagram



Pending request register = a flag that stays on meaning "this line has a pending interrupt"

It signals the NVIC, which stops the CPU and runs the ISR

The flag **must be cleared** inside the ISR, otherwise the interrupt fires forever.

It is cleared by writing 1 to it.

The **event** path does not run code and **does not set a pending flag**.

Through the Event mask register → Pulse generator → a short hardware pulse.

Used to wake the chip from low-power mode or trigger another peripheral.

Nothing to clear afterwards.

EXTI register map

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	**EXTI_IMR**	Reserved									MR [22:21]		Reserved		MR[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	**EXTI_EMR**	Reserved									MR [22:21]		Reserved		MR[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x08	**EXTI_RTSR**	Reserved									TR [22:21]		Reserved		TR[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x0C	**EXTI_FTSR**	Reserved									TR [22:21]		Reserved		TR[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x10	**EXTI_SWIER**	Reserved									SWIER [22:21]		Reserved		SWIER[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x14	**EXTI_PR**	Reserved									PR [22:21]		Reserved		PR[18:0]																		
	Reset value										0	0			0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Mask registers (IMR / EMR) → enable interrupt / event per line.

Trigger registers (RTSR / FTSR) → select rising / falling edge per line

SWIER → trigger a line by software

PR → pending flag (cleared by writing 1)

Bits 0–15 = **GPIO lines**; bits 16–18 and 21-22 = **internal lines** (PVD, RTC, USB)

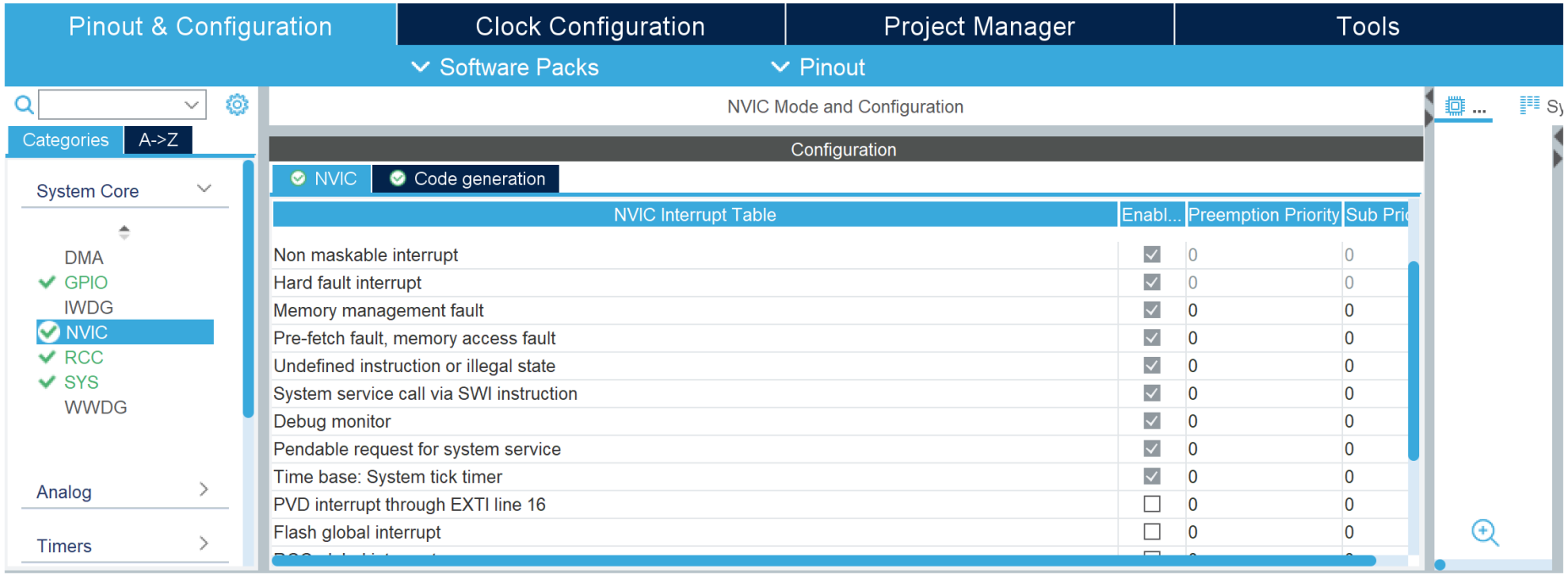
Reset value = all 0 → nothing enabled at startup; you turn on only the bits you need

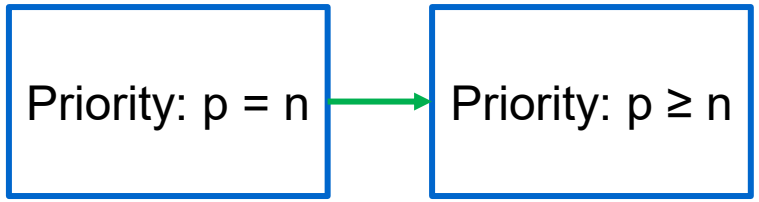
Interrupt vector table

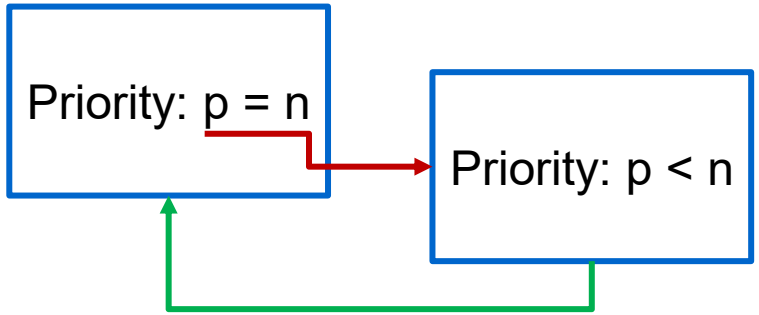
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
24	31	settable	TIM1_BRK_TIM9	TIM1 Break interrupt and TIM9 global interrupt	0x0000 00A0
25	32	settable	TIM1_UP_TIM10	TIM1 Update interrupt and TIM10 global interrupt	0x0000 00A4
26	33	settable	TIM1_TRG_COM_TIM11	TIM1 Trigger and Commutation interrupts and TIM11 global interrupt	0x0000 00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000 00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000 00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000 00B8
31	38	settable	I2C1_EV	I^2C1 event interrupt	0x0000 00BC
32	39	settable	I2C1_ER	I^2C1 error interrupt	0x0000 00C0
33	40	settable	I2C2_EV	I^2C2 event interrupt	0x0000 00C4
34	41	settable	I2C2_ER	I^2C2 error interrupt	0x0000 00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000 00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000 00D0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0

- The **vector table** is an array of pointers located at a fixed memory address in the microcontroller (specifically in the code portion).
- Each entry in the table is the **starting address** of an interrupt handler function (ISR).
- When an interrupt occurs, the processor uses the vector table to jump to the corresponding ISR, executing the specific routine for that interrupt.

Interrupt vector table







3. Functions and exercises

03

GPIO HAL functions

GPIO_PinState **HAL_GPIO_ReadPin**(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)

void **HAL_GPIO_WritePin**(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)

void **HAL_GPIO_TogglePin**(GPIO_TypeDef* GPIOx, uint16_t GPIO_Pin)

___weak void **HAL_GPIO_EXTI_Callback**(uint16_t GPIO_Pin)
Function called in the interrupt routine (after flag reset), it can be redefined by user in main.c

GPIO HAL functions

- Interrupts must be enabled in CUBE
- All the interrupt routines are handled in stm32f4xx_it.c

Project 1a: Pushbutton - polling

- Objective of this project is to **switch on the green LED** on Nucleo board (LD2), **every time the blue pushbutton is pressed** and to **switch it off when the pushbutton is released**.
- A **polling operation** will be used to monitor the state of the pushbutton.

Project 1a hints

- Use the NUCLEO board Manual to find the GPIO connected to the blue push-button and LD2.
- Configure the GPIO in CUBE and generate the code.
- In NUCLEO board Manual find the state of the blue button (0 or 1) when it is pressed.
- Use the GPIO function the check the state of the push-button and consequently update the state of LD2.
- Debug and verify if the LED is ON when you press it and OFF when you release it.

Project 1b: Pushbutton - interrupt

- Objective of this project is to switch on an LED every time the blue pushbutton is pressed, and to switch it off when the pushbutton is released.
- The pushbutton input will be used in **interrupt mode**.

Project 1b hints

- Configure CUBE, remembering to enable the interrupt for the pushbutton. Note that you need to modify the status of the LED both when you press and when you release the pushbutton.
- Check in the **stm32f4xx_it.c** file that the routine which handles the EXTI from the pushbutton has been automatically prepared by CUBE.
- Use the **HAL_GPIO_EXTI_Callback** to manage the interrupt (switch ON/OFF the LED), using the GPIO functions.
- Debug and verify if the LED is ON when you press it and OFF when you release it. You should see the same behavior you had in the first project (polling),
but now you are not using CPU resources just to wait for an event!



POLITECNICO MILANO 1863 | DEPARTMENT OF ELECTRONICS INFORMATION AND BIOENGINEERING

Arianna.devecchi@polimi.it
Federica.villa@polimi.it
www.polimi.it

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA